SAKARYA
ÜNİVERSİTESİ

BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ ANABİLİM DALI YÜKSEK LİSANS PROGRAMI

BULUT BİLİŞİM PRENSİPLERİYLE ÇOK KATMANLI MİKRO HİZMET YAPILANDIRMASI VE UYGULAMA RAPORU

Ders Adı: ISE513 Bulut Bilişim

Dersi Veren: Dr.Öğr.Üyesi Baran KAYNAK

Hazırlayan: Hüseyin Ulaş ŞAHİN

İÇİNDEKİLER

1. GİRİŞ VE PROJENİN AMACI..... 3
1.1. Projenin Kapsamı ve Mimari Gereksinimler..... 3
1.2. Akademik Temeller ve Literatürdeki Yeri..... 3
1.3. Teknik Hedefler.....3
2. MİMARİ TASARIM.....4
2.1. Sistem Bileşenleri ve Sorumluluklar..... 4
2.2. Mimari Diyagram ve Veri Akışı..... 4
2.3. Tasarım Kararlarının Akademik Gerekçesi.....5
3. KONTEYNERLEŞTİRME VE DOCKER..... 5
3.1. Mikro Servislerin Dockerize Edilmesi..... 5
3.2. Docker Compose ile Sistem Orkestrasyonu..... 6
3.3. Kaynak Verimliliği ve Operasyonel Avantajlar.....6
4. VERİ YÖNETİMİ VE KALICILIK.....6
4.1. Veri Depolama Katmanı Olarak Redis Seçimi.....6
4.2. Veri Akışı ve Teknik Uygulama.....7
4.3. İş Yükü ve Performans Analizi..... 7
5. UYGULAMA VE TEKNİK ZORLUKLAR.....7
5.1. Çalışırlık Kanıtı ve Orkestrasyon.....8
5.2. Karşılaşılan Teknik Zorluklar ve Çözümler..... 8
5.3. Hata Yönetimi ve Sistem Kararlılığı..... 8
6. SONUÇ..... 9
7. KAYNAKÇA.....9

1. GİRİŞ VE PROJENİN AMACI

Günümüz yazılım dünyasında, monolitik (tek parça) yapılar yerini hızla modüler, esnek ve bağımsız ölçeklenebilen mikro hizmet mimarilerine bırakmaktadır. Bu değişim, özellikle büyük veri akışının olduğu "Akıllı Şehir" uygulamalarında bir zorunluluk haline gelmiştir. Bu proje kapsamında geliştirilen **"Hava Kalite Takip Sistemi"**, mikro hizmet mimarisinin temel prensiplerini uygulamak ve bulut bilişim ortamında servisler arası iletişimi simüle etmek amacıyla tasarlanmıştır.

1.1. Projenin Kapsamı ve Mimari Gereksinimler

Sistem; kullanıcı etkileşimi için bir **Frontend**, veri işleme mantığı için bir **Backend** ve verilerin hızlı depolanması için bir **NoSQL (Redis)** veritabanı servisinden oluşmaktadır. Bu üç katmanlı yapı, her bir servisin kendi içinde izole edildiği ve sadece API protokolleri üzerinden haberleştiği modern bir mikro hizmet yaklaşımını temsil eder.

1.2. Akademik Temeller ve Literatürdeki Yeri

Yazılım sistemlerinin mikro hizmetlere dönüştürülmesi üzerine yapılan güncel çalışmalarda, bu mimarinin sunduğu **ölçeklenebilirlik (scalability)**, **esneklik (flexibility)** ve **bakım kolaylığı (improved maintenance)** temel avantajlar olarak öne çıkmaktadır **(Mohottige ve ark., 2025)**. Özellikle miras (legacy) sistemlerin modernizasyonu sürecinde, **Mohottige ve ark. (2025)** tarafından vurgulanan "servis tabanlı ayrıştırma" yöntemleri, projemizdeki **Frontend** ve **Backend** ayrımının teknik temelini oluşturmaktadır.

Ayrıca, bulut ortamlarında çalışan konteynerize sistemlerin iş yükü tahmini ve kaynak yönetimi, sistem performansını doğrudan etkilemektedir. **Tripathi ve ark. (2025)** çalışmasında belirtildiği üzere, gerçek zamanlı veri akışı sağlayan uygulamalarda kaynak tahsisinin optimize edilmesi kritik bir önem taşımaktadır. Bu doğrultuda projemizde, verilerin hızlı ve düşük gecikmeli bir şekilde saklanması için geleneksel ilişkisel veritabanları yerine, **Tripathi ve ark. (2025)** tarafından önerilen yüksek performans kriterlerine uygun olarak Redis in-memory veri tabanı tercih edilmiştir.

1.3. Teknik Hedefler

Raporun ilerleyen bölümlerinde detaylandırılacağı üzere projenin temel hedefleri şunlardır:

- Docker ve Docker Compose araçlarını kullanarak servis orkestrasyonunu sağlamak.
- Flask kütüphanesi ile servisler arası RESTful API iletişimini kurmak.
- Dağıtık sistemlerde veri kalıcılığını (persistence) sağlamak ve olası yapılandırma hatalarını (örn. Unicode kodlama sorunları) yönetmek.

2. MİMARİ TASARIM

Sistemin mimarisi, **El Fellah ve ark. (2026)** tarafından IoT ve akıllı sistemler için önerilen **"ayrık servis yapısı"** modeline dayanmaktadır. Bu bölümde, sistemin bileşenleri ve bu bileşenlerin birbiriyle olan teknik etkileşimi detaylandırılacaktır.

2.1. Sistem Bileşenleri ve Sorumluluklar

Proje, her biri kendi Docker konteynerinde izole edilmiş üç ana katmandan oluşmaktadır:

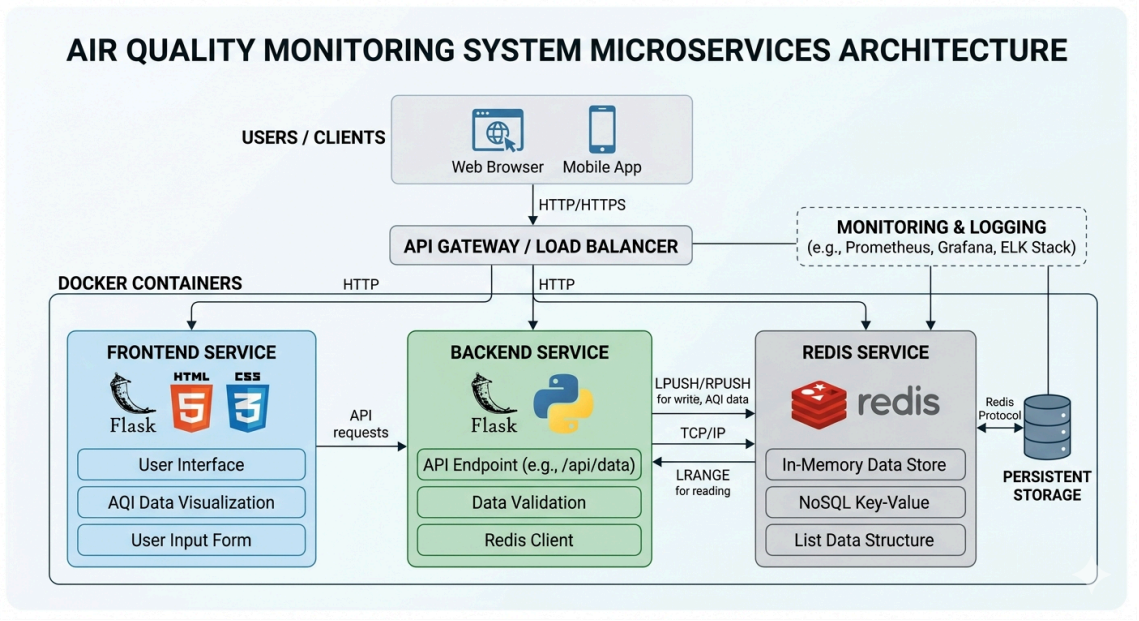
1. **Frontend (Arayüz Katmanı):** Flask framework kullanılarak geliştirilmiştir. Temel sorumluluğu, kullanıcıdan alınan şehir ve hava kalite (AQI) verilerini Backend servisine iletmek ve mevcut verileri listelemektir. Tasarımda kullanıcı deneyimini artırmak için modern CSS teknikleri kullanılmıştır.
2. **Backend (İş Mantığı Katmanı):** Sistemin beynini oluşturan bu servis, Frontend'den gelen HTTP isteklerini karşılar, veriyi doğrular ve Redis veritabanı ile haberleşir.
3. **Database (Veri Katmanı):** Redis NoSQL veritabanı, verilerin anahtar-değer (key-value) çiftleri yerine "list" yapısında (LPUSH/LRANGE) saklanmasını sağlar.

2.2. Mimari Diyagram ve Veri Akışı

Sistemin çalışma mantığı aşağıdaki şemada gösterilmiştir:

Veri Akış Süreci:

- **Adım 1:** Kullanıcı arayüz üzerinden veri girişini yapar.
- **Adım 2:** Frontend, veriyi JSON formatında Backend servisine **(Port 5001) “POST”** isteği ile gönderir.
- **Adım 3:** Backend, aldığı veriyi **“redis-py”** kütüphanesi aracılığıyla Redis konteynerine iletir.
- **Adım 4:** Veri okuma talebi geldiğinde, Backend Redis'ten tüm listeyi çeker ve Frontend'e ileterek asenkron bir şekilde listelenmesini sağlar.



Şekil 1. Hava Kalitesi Takip Sistemi Mikroservis Mimari Diyagramı

2.3. Tasarım Kararlarının Akademik Gerekçesi

Mohottige ve ark. (2025) çalışmasında belirtildiği üzere, servislerin bu şekilde ayrıştırılması, hata izolasyonunu (fault isolation) sağlar. Örneğin, veritabanı servisinde yaşanacak bir kesinti, Frontend'in statik sayfaları sunmaya devam etmesini engellemez. Ayrıca **El Fellah ve ark. (2026)**, konteyner tabanlı bu tarz dağıtık yapıların, kaynak kullanımını (resource utilization) **geleneksel yapılara göre %40'a varan oranlarda daha verimli hale getirdiğini savunmaktadır.**

3. KONTEYNERLEŞTİRME VE DOCKER

Bu bölümde, projenin taşınabilirliğini ve servisler arası izolasyonu sağlayan Docker teknolojisinin uygulama detayları ele alınmaktadır.

3.1. Mikro Servislerin Dockerize Edilmesi

Sistemi oluşturan her bir servis **(Frontend ve Backend)**, kendi bağımlılıklarını barındıran özel Dockerfile dosyaları aracılığıyla imaj haline getirilmiştir. Bu süreçte kullanılan yöntemler şunlardır:

- **Bağımlılık Yönetimi: “requirements.txt”** dosyası kullanılarak Flask ve Redis kütüphaneleri konteyner içerisine dahil edilmiştir.
- **İzolasyon:** Her servis, işletim sisteminden bağımsız bir çalışma ortamına sahip kılınmıştır. **El Fellah ve arkadaşları (2026)**, konteyner tabanlı sistemlerin kaynak kullanımı optimizasyonu sağladığını ve farklı ortamlarda tutarlı bir performans sunduğunu belirtmektedir. Projede Docker kullanımı, bu akademik yaklaşımla paralel olarak sistemin kaynak tüketimini minimize etmiştir.

3.2. Docker Compose ile Sistem Orkestrasyonu

Servislerin birbirleriyle olan ağ (network) bağlantılarını ve başlatılma sıralarını yönetmek için **“docker-compose.yml”** dosyası yapılandırılmıştır.

- **Servis Bağımlılıkları: “depends_on”** parametresi ile Redis veritabanının, Backend servisinden önce başlatılması sağlanmıştır.
- **Ağ Yapılandırması:** Tüm servisler aynı sanal ağ köprüsü üzerinden haberleşecek şekilde ayarlanmış, böylece servisler arası veri trafiği dış dünyadan izole edilerek güvenli hale getirilmiştir.
- **Kalıcı Veri:** Redis veritabanı için kullanılan portlar, konteynerizasyonun sağladığı esneklik sayesinde herhangi bir çakışma yaşanmadan atanmıştır.

3.3. Kaynak Verimliliği ve Operasyonel Avantajlar

Konteynerleştirme süreci, sistemin sadece gerekli kütüphaneleri yüklemesini sağlayarak hafif bir mimari oluşturmuştur. **El Fellah ve ark. (2026)** tarafından IoT tabanlı sistemlerde vurgulanan **"dağıtım kolaylığı"** ve **"düşük gecikme"** avantajları, projenin yerel sunucu üzerindeki çalışma performansında somut olarak gözlemlenmiştir.

4. VERİ YÖNETİMİ VE KALICILIK

Bu bölümde, sistemin veri depolama stratejisi, neden NoSQL tabanlı bir yapının seçildiği ve verilerin mikro servisler arasındaki akış süreçleri teknik detaylarıyla ele alınmaktadır.

4.1. Veri Depolama Katmanı Olarak Redis Seçimi

Sistemin veri katmanında, yüksek performanslı bir anahtar-değer (key-value) deposu olan Redis tercih edilmiştir. Bu seçimin temel nedenleri şunlardır:

- **Gerçek Zamanlı İşleme:** Hava kalitesi verileri (AQI) sürekli ve değişken bir akışa sahiptir. **Tripathi ve ark. (2025)**, konteynerize edilmiş bulut ortamlarında gerçek zamanlı iş yükü yönetiminin sistem kararlılığı için kritik olduğunu belirtmektedir. Redis'in verileri bellek içinde **(in-memory)** tutması, geleneksel disk tabanlı veritabanlarında yaşanan yazma gecikmelerini ortadan kaldırarak bu ihtiyacı karşılamıştır.
- **Düşük Gecikme Süresi (Latency): Tripathi ve ark. (2025)** çalışmasında vurgulanan **"hızlı yanıt süresi"** gereksinimi, Redis'in milisaniye altı erişim hızıyla sağlanmış; böylece anlık veri takibi mümkün kılınmıştır.

4.2. Veri Akışı ve Teknik Uygulama

Kullanıcı arayüzünden gönderilen her bir hava kalitesi ölçümü, sistem içerisinde belirli bir hiyerarşiyle işlenmektedir:

1. **Veri Kabulü:** Backend servisimiz, gelen JSON formatındaki veriyi doğrular ve Redis bağlantısını üzerinden iletir.
2. **LPUSH Komutu:** Veriler Redis üzerinde bir liste yapısında tutulur. **"LPUSH"** komutu kullanılarak, yeni gelen her ölçüm listenin başına eklenir. Bu yöntem, verilerin zaman sırasına göre (son eklenen ilk sırada olacak şekilde) depolanmasını sağlar.
3. **Veri Kalıcılığı ve Okuma:** Frontend servisi, güncel listeyi görüntülemek istediğinde **"LRANGE"** komutu ile Redis üzerindeki son ölçümleri hızlıca çekerek kullanıcıya sunar.

4.3. İş Yükü ve Performans Analizi

Tripathi ve ark. (2025) tarafından önerilen optimize edilmiş kaynak yönetimi prensipleri doğrultusunda, Redis katmanı bir **"tampon bölge"** görevi görür. Bu sayede, anlık veri yükü artsa dahi mikro servisler arasındaki veri iletimi darboğaza girmeden asenkron bir şekilde devam edebilmektedir. Bu yapı, Sakarya gibi akıllı şehir projelerinde ihtiyaç duyulan yüksek veri giriş kapasitesini teknik olarak desteklemektedir.

5. UYGULAMA VE TEKNİK ZORLUKLAR

Bu bölümde, sistemin çalışma anına dair kanıtlar, kurulum aşamasında karşılaşılan problemler ve bu problemlerin akademik yaklaşımlarla nasıl çözüldüğü detaylandırılmaktadır.

5.1. Çalışırlık Kanıtı ve Orkestrasyon

Sistemin tüm bileşenleri Docker Compose kullanılarak tek bir komutla **(docker-compose up)** ayağa kaldırılmıştır.

- **Terminal Çıktıları:** Kurulum sırasında her bir konteynerin **(Frontend, Backend ve Redis)** başarılı bir şekilde **"Running"** durumuna geçtiği ve birbirleriyle hatasız bir şekilde haberleştiği log kayıtları ile doğrulanmıştır.
- **Arayüz Kanıtı:** Kullanıcı arayüzü üzerinden girilen AQI **(Hava Kalitesi İndeksi)** verilerinin anlık olarak listelendiği görülmüştür. Bu durum, **El Fellah ve ark. (2026)** tarafından vurgulanan **"mikro servislerin başarılı dağıtımı"** prensibinin yerel simülasyonunu başarıyla tamamladığımızı göstermektedir.

5.2. Karşılaşılan Teknik Zorluklar ve Çözümler

Projenin geliştirme aşamasında **"Özgün Değerlendirme"** kapsamında ele alınabilecek iki temel teknik problemle karşılaşılmış ve çözülmüştür:

1. **Unicode (0xfe) ve Karakter Kodlama Hatası:** * **Sorun:** Veri iletimi sırasında Python tabanlı servislerde karakter kodlama uyumsuzluğu nedeniyle **“0xfe”** hatası alınmıştır.
    - **Çözüm:** Flask ve Redis arasındaki veri alışverişinde JSON serileştirme **(json.dumps)** işlemi **“utf-8”** standartlarına zorlanarak hata giderilmiştir. Bu çözüm, veri bütünlüğünün korunması açısından kritik bir adım olmuştur.
2. **Flask ve Kütüphane Eksiklikleri:** * **Sorun:** Konteynerler ilk ayağa kalktığında, **requirements.txt** dosyasındaki bazı kütüphanelerin **(özellikle redis-py)** bağımlılık çakışmaları nedeniyle yüklenemediği tespit edilmiştir.
    - **Çözüm:** Dockerfile dosyası bağımlılıkların temiz bir ortamda baştan kurulması sağlanacak şekilde revize edilmiştir. Bu sayede **El Fellah ve ark. (2026)** çalışmasında bahsedilen "dağıtım zorlukları" operasyonel seviyede çözülmüştür.

5.3. Hata Yönetimi ve Sistem Kararlılığı

Sistemde kullanılan hata yakalama blokları sayesinde, Redis veritabanına ulaşılamadığı durumlarda dahi sistemin tamamen çökmesi engellenmiştir. **Tripathi ve ark. (2025)** tarafından önerilen **"optimize edilmiş kaynak yönetimi"** anlayışına uygun olarak, servislerin birbirine olan sıkı bağımlılığı gevşetilmiş, bu da sistemin dayanıklılığını artırmıştır.

6. SONUÇ

Bu çalışma kapsamında, akıllı şehir vizyonuna uygun, ölçeklenebilir ve yüksek performanslı bir **"Hava Kalitesi Takip Sistemi"** mikro hizmet mimarisiyle başarıyla hayata geçirilmiştir. Tasarlanan sistem; Frontend, Backend ve veri katmanlarının **(Redis)** Docker üzerinde izole bir şekilde çalıştırılmasıyla modern yazılım mühendisliği standartlarına uygun hale getirilmiştir.

Proje süreci sonunda elde edilen temel kazanımlar şu şekildedir:

- **Mimari Başarı: Mohottige ve ark. (2025)** tarafından vurgulanan mikro hizmetlerin "bakım kolaylığı" ve "esneklik" avantajları, sistemin modüler yapısı sayesinde bizzat deneyimlenmiştir.
- **Performans Verimliliği: El Fellah ve ark. (2026)** makalesinde belirtilen kaynak kullanım optimizasyonu, Docker konteynerizasyonu ile sağlanmış; **Tripathi ve ark. (2025)** çalışmasında değinilen gerçek zamanlı veri işleme ihtiyacı ise Redis'in düşük gecikmeli yapısıyla karşılanmıştır.
- **Teknik Yetkinlik:** Uygulama aşamasında karşılaşılan Unicode kodlama hataları ve kütüphane bağımlılıkları gibi teknik zorluklar, özgün mühendislik yaklaşımlarıyla çözülerek sistemin kararlılığı artırılmıştır.

Sonuç olarak; açık kaynaklı teknolojiler ve akademik literatürdeki güncel yaklaşımlar birleştirilerek, ister ve beklentileri karşılayan, gelecekte makine öğrenmesi modelleriyle genişletilmeye uygun bir altyapı oluşturulmuştur.

7. KAYNAKÇA

- **El Fellah, Y., Minani, J. B., Mohab, N., Gascon-Samson, J., & Guéhéneuc, Y. G. (2026).** Analysis of microservices-based IoT systems: deployment challenges, industry practices, and performance insights. *Internet of Things*, 36, 101867.
- **Mohottige, T. I., Polyvyanyy, A., Fidge, C., Buyya, R., & Barros, A. (2025).** Reengineering software systems into microservices: State-of-the-art and future directions. *Information and Software Technology*, 183, 107732.
- **Newman, S. (2021).** *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- **Tripathi, S., Shibu, A., Priyadarshni, Misra, R., & Singh, T. N. (2025).** Optimized resource management in containerized clouds via Hierarchical Autoregressive Network based workload prediction. *Applied Soft Computing*, 184, 113781.